THE CHALLENGE

# Ship or Block

An AI that answers legal questions is only as good as the answers it refuses to give.

## The story

Azta answers tax and legal questions for accountants and advisors across 28 countries. Every answer is generated by a language model and cites the exact paragraphs of law it relies on.

Most answers are right. Some are not, and the wrong ones look exactly like the right ones: a number from an old version of a law, an exception hidden in another act, a definition that changes who a rule applies to, a sentence the model made up. One confident wrong answer can cost a client real money.

So before an answer reaches a user, something has to check it. Today, you build that something.

YOUR TASK

Build a program that checks 500 AI-drafted answers against the law of a fictional country as it stood on the day each question was asked. For each one, decide: **SHIP** it to the user, or **BLOCK** it and point to the provision that proves it wrong. You have $10 of AI credit.

## What you get

### The law of Nordvale

A complete, made-up legal system in plain English: about 430 acts and 21,000 section versions on everyday fees, permits and allowances, with every version each section has ever had. No legal knowledge needed. It is far too big to paste into any model.

`Drive: corpus/acts.json, corpus/provisions.jsonl`

### The answers

100 practice cases with the right decision and, for wrong answers, the provision that proves it. No explanations: working out why is part of the job. 500 test cases without labels: these are what you are scored on.

`Drive: practice.jsonl, test.jsonl, starter/`

**500**
answers to judge

**$10**
AI credit per team

**70 min**
from briefing to submission

# How the law looks

Every line of `provisions.jsonl` is one version of one section. Sections change over time, and some sections change what others say.

> `FDA §14 · Fees and Duties Act · version 1 · in force 2024-01-01 → 2026-07-01`
>
> (1) The owner of an electric bicycle shall pay an annual registration fee of 24 crowns. (2) An owner under 18 years of age is exempt … (4) This section does not apply to an electric bicycle used exclusively for commercial deliveries …

> `FDA §14 · version 2 · in force 2026-07-01 → (no end)`
>
> (1) The owner of an electric bicycle shall pay an annual registration fee of **30** crowns. …

> `SSA §7 · Student Support Act · in force 2024-09-01 → (no end)`
>
> Notwithstanding FDA §14 and FDA §22, a full-time student pays half of the electric bicycle registration fee and half of the dog licence fee.

A version is in force on day D when `in_force_from ≤ D < in_force_to`. On 2026-07-01 version 2 applies, not version 1.

# Three answers

`asked_on 2026-03-10 · cites FDA §14(1)`

**SHIP**

**Q** I'm 25 and I own an e-bike. What is the yearly registration fee?

**A** The annual registration fee for an electric bicycle is 24 crowns.

Correct for the version in force in March 2026.

`asked_on 2026-09-02 · cites FDA §14(1)`

**BLOCK**

**Q** I'm 25 and I registered my new e-bike last week. What is the yearly registration fee?

**A** The annual registration fee for an electric bicycle is 24 crowns.

Same answer, different day. Since 2026-07-01 the fee is 30 crowns. Evidence: `FDA §14`.

`asked_on 2026-09-02 · cites FDA §14(1)`

**BLOCK**

**Q** I'm a 25-year-old full-time student and I registered my new e-bike last week. What do I pay per year?

**A** The annual registration fee for an electric bicycle is 30 crowns.

Everything the answer cites is true, and it is still wrong: SSA §7, in another act, halves the fee for students. Evidence: `SSA §7`.

## When is an answer correct?

SHIP an answer only if all four hold on the law in force on `asked_on`. Otherwise, BLOCK it.

1. Every statement in the answer is supported by the provisions it cites.
2. Every citation exists, was in force on `asked_on`, and supports the statement it backs.
3. No other provision in force on `asked_on` changes the answer for the facts given in the question: no exception, override, definition, transitional rule or special rate is missed.
4. Simple arithmetic from the law counts as supported. "Half of 30 crowns is 15 crowns" is fine.

Facts the question does not state are not assumed. If the question doesn't say the person is a student, the student rule does not make the answer wrong. A statement that no reduction or other rule applies is correct exactly when rule 3 holds. The law uses formal defined terms, and a definition applies across acts.

## The data

```
# test.jsonl: one case per line
{"case_id": "T001", "asked_on": "2026-09-02",
 "question": "I'm 25 and I registered my new e-bike last week. What is the yearly fee?",
 "answer": "The annual registration fee for an electric bicycle is 24 crowns.",
 "citations": ["FDA §14(1)"]}

# practice.jsonl: the same, plus the truth (no explanation)
 "label": "BLOCK", "evidence": ["FDA §14"]

# corpus/provisions.jsonl: one section version per line
{"provision_id": "FDA §14", "act_id": "FDA", "section": 14, "heading": "Electric bicycle
registration fee",
 "version": 2, "in_force_from": "2026-07-01", "in_force_to": null, "text": "(1) The owner of
..."}
```

## Starter kit

The `starter/` folder in the Drive has plumbing, no solution: loading the data, model calls with retries and cost tracking, `python budget.py` for your remaining credit, `python score.py` to score yourself on the practice set with the official rule, and `python submit.py` to check your file before sending. Your key works with any OpenRouter model. Your $10 is a safety limit, not part of the score; when it is gone, the key stops working. Develop on the practice set and run the test set once you are happy: a full test run costs roughly $1 to $3 depending on the model. What one answer costs in your final system is part of the jury's feasibility score, so know your number.

```
# export OPENROUTER_API_KEY=sk-or-...   (Python 3.9+, no packages needed)
from load import load_corpus, load_cases
from llm import chat, run_parallel, total_cost

acts, provisions = load_corpus("corpus")        # {"FDA §14": [version 1, version 2, ...]}
cases = load_cases("test.jsonl")
reply = chat("...", model="google/gemini-2.5-flash-lite")   # any model id from
openrouter.ai/models
print(reply.text, reply.cost, total_cost())
```

# What you submit

`9:40` start · **`10:50` hard deadline**, keys switched off · `10:55` pitches. Results are announced later in the day.

By 10:50, upload `submission.csv`, your code (zip or repository link) and at most 2 slides (PDF) to **your team's Google Drive folder** (you get its link with your key). Only files uploaded before 10:50 count.

```
case_id,decision,evidence
T001,SHIP,
T002,BLOCK,SSA §7
...
```

One evidence provision per row, at section level (`SSA §7`, not `SSA §7(1)`). If several provisions prove the answer wrong, any one of them counts. Missing or invalid rows count as BLOCK without evidence.

# Scoring

| YOUR DECISION | ANSWER WAS CORRECT | ANSWER WAS WRONG |
|---|---|---|
| **SHIP** | +1 | −3 |
| **BLOCK** with the right evidence | −1 | +1 |
| **BLOCK** with wrong or no evidence | −1 | 0 |

```
score = sum of the points in the table   (maximum 500)
```

Shipping a wrong answer costs three times what a right one earns, and blocking a good answer costs a point; a block only scores with the provision that proves it wrong. **Final points (100):** 70 from the score (best team gets 70, others in proportion; negative scores get 0), 30 from the jury's assessment of your system.

# Your system: 2-minute pitch, hard stop

Show the jury how your gate works, which wrong answers it catches and which slip through, and where your money went. **Strictly 2 slides**; extra slides are cut. No questions. The jury scores your system (30): **real-life feasibility** (10: what does one answer cost, and would it hold up at real scale and speed?), **architecture and reasoning** (10), **knowing what it misses** (5) and **the pitch itself** (5).

# Rules

- Every decision must come from your program. No hand-labelling of test cases. Hand-written rules and code are fine.
- Models that judge cases run through your issued key or on your own laptop. No other API keys, accounts or subscriptions for judging.
- AI coding assistants may help you write code. They may not read the test cases and decide them for you.
- `:free` models on OpenRouter are not allowed (their limits are shared by all teams).
- The jury may ask you to show your pipeline running.